

Proyecto 2: Memory Mappings en Selfie

Profesores Jorge Gonzalez, Mauricio Pinto
TAs Mariana Capuñay

1 Implementar la syscall `mmap` en Selfie

La syscall `mmap` permite crear un mapping entre una región de memoria virtual de un proceso y una región de un archivo. En esta implementación simplificada para Selfie, el mapping deberá asociar un rango de direcciones virtuales del proceso con páginas físicas almacenadas en el *page cache*. Estas páginas físicas contendrán el contenido del archivo a partir de un determinado offset. De esta manera agregamos la posibilidad de que múltiples procesos compartan acceso a un archivo sin necesidad de acceder a memoria secundaria. Esta relación deberá registrarse en el contexto del proceso mediante un atributo `mappings`. Cada entrada de `mappings` describirá un rango de direcciones virtuales del proceso y el archivo asociado a dicho rango.

Para este proyecto, se debe considerar el tamaño de página utilizado por Selfie, definido por `PAGESIZE`. En la implementación base, `PAGESIZE` corresponde a 4096 bytes. Este valor deberá usarse para calcular la cantidad de páginas necesarias, alinear direcciones y validar offsets.

La llamada propuesta para Selfie puede definirse usando valores de tipo `uint64_t`:

```
uint64_t mmap(uint64_t addr, uint64_t length, uint64_t prot, uint64_t fd, uint64_t offset);
```

Donde:

- `addr`: dirección virtual inicial donde se desea crear el mapping. Si `addr = 0`, Selfie puede elegir una región virtual libre.
- `length`: cantidad de bytes que se desea mapear. Internamente, este valor puede redondearse hacia arriba al múltiplo de `PAGESIZE` más cercano.
- `prot`: permisos del mapping. Por ejemplo, 0 para lectura, 1 para escritura y 2 para lectura/escritura.
- `fd`: file descriptor (identificador) del archivo cuyo contenido será mapeado.

Nota

En Selfie, el *file descriptor* (`fd`) es un identificador local al proceso. Por ello, un mismo valor de `fd` no identifica globalmente al mismo archivo en todos los procesos. Dos procesos podrían tener valores distintos de `fd` para referirse al mismo archivo, o incluso el mismo valor de `fd` para referirse a archivos diferentes.

- `offset`: posición inicial dentro del archivo desde donde comenzará el mapping. Se recomienda que sea múltiplo de `PAGESIZE`.

Importante

La syscall `mmap` no debe escribir cambios en el archivo al momento de crear el mapping. Su función principal es crear la relación entre memoria virtual, tabla de páginas y *page cache*.

Los cambios realizados por el proceso sobre la memoria mapeada se almacenan inicialmente en los *cache frames*. Para este proyecto, dichos cambios se escribirán en el archivo únicamente cuando el proceso invoque `msync`. Para comparar esta implementación simplificada con el comportamiento real en Linux, se recomienda revisar la documentación oficial de `mmap`, la cual está disponible en <https://man7.org/linux/man-pages/man2/mmap.2.html>.

Importante

Para este proyecto, **considerar que cada test trabajará con mappings asociados a un único archivo.**

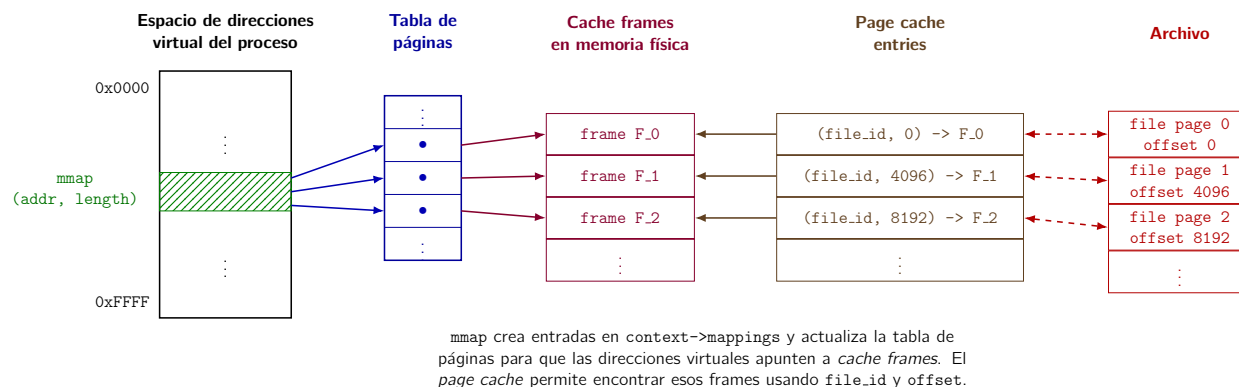


Figure 1: Relación entre memoria virtual, tabla de páginas, *page cache*, frames físicos y archivo en una implementación simplificada de `mmap`.

2 Implementar `syscall munmap` en Selfie

La `syscall munmap` elimina un mapping previamente creado. Para este proyecto, deberá definirla como:

```
uint64_t munmap(uint64_t addr);
```

A diferencia de Linux, esta versión simplificada no recibe `length`. Por ello, `addr` debe coincidir con la dirección virtual inicial de un mapping previamente registrado en `context->mappings`.

En Linux, `munmap` recibe tanto la dirección inicial como el tamaño del rango a desmapear; por lo tanto, se recomienda revisar la documentación oficial (disponible en <https://man7.org/linux/man-pages/man2/mmap.2.html>) para entender la diferencia con la versión reducida propuesta en este proyecto.

En esta versión simplificada de Selfie, `munmap` no debe escribir los cambios en el archivo asociado. Su función será eliminar la relación entre el rango de memoria virtual del proceso y las páginas físicas del *page cache*. Por lo tanto, si el proceso modifica una región mapeada y luego invoca únicamente `munmap`, dichos cambios no deberán persistir en el archivo. La escritura de cambios al archivo deberá realizarse únicamente mediante `msync`.

3 Implementar `msync` en Selfie

La `syscall msync` debe permitir que los cambios realizados sobre una región mapeada en memoria se escriban de vuelta en el archivo asociado. En este proyecto, `msync` será el único mecanismo encargado de persistir en el archivo los cambios hechos sobre una región mapeada.

Nota

Pueden utilizar la `syscall write` para realizar la actualización del archivo o crear una nueva `syscall msync`.

En Linux, `msync` sincroniza con el sistema de archivos los cambios hechos sobre una región previamente mapeada con `mmap`. Una versión simplificada de esta `syscall` en Selfie es:

```
uint64_t msync(uint64_t addr);
```

En esta versión, `addr` debe coincidir con la dirección virtual inicial de un mapping previamente registrado en `context->mappings`. A partir de esta dirección, Selfie debe identificar el archivo asociado, el `offset` inicial y el tamaño del mapping.

Para comparar esta implementación simplificada con el comportamiento real en Linux, se recomienda revisar la documentación oficial de `msync`: <https://man7.org/linux/man-pages/man2/msync.2.html>

4 Pasos sugeridos

1. Agregar un atributo `mappings` al contexto; para cada mapping debe guardar como mínimo:
 - Address inicial para mapear el page virtual
 - Offset desde el que se hace el mapping del archivo. *Se recomienda que el offset sea divisible por `PAGESIZE`.*
 - Tamaño del mapping.
 - File descriptor o identificador del archivo asociado al mapping.
2. Crear un espacio reservado para los *cache frames*. Podemos crear un bloque de memoria separado de la memoria principal de *selfie*. Para ello, guiarnos de la función `init_memory`.
3. Crear una lista de entries del *page cache*. Para ello podemos:
 - Crear un bloque de memoria fijo de un tamaño razonable para almacenar *entries*.
 - Crear una lista enlazada para almacenar *entries*.

Cada *entry* debe contener la información ilustrada en la Tabla 4

Hint

Puedes revisar la implementación de las syscalls `read` o `write` para comprender la lectura y escritura de archivos desde *Selfie*.

Nota

Un *cache frame* es una página física. A diferencia de las páginas virtuales, los *cache frames* no pertenecen a un proceso específico. Estos pueden ser compartidos por varios procesos cuando sus mappings apuntan a la misma página de un archivo. Para identificar qué contenido almacena cada *cache frame*, el *page cache* mantiene entradas asociadas al archivo y al offset de página correspondiente.

File ID	File page offset	Cache frame
23	0	P0
23	4096	P1
23	8192	P2

Table 1: Ejemplo de entradas almacenadas en el *page cache*.

Cada entrada del *page cache* indica qué *cache frame* contiene una página específica del archivo. **Ese frame puede ser compartido por varios procesos si sus mappings apuntan al mismo archivo y al mismo offset de página.**

Nota

Pueden mapear el archivo completo, y en el mapping de cada proceso sólo mantener una referencia a la porción mapeada por ese proceso.

Al acceder a una dirección presente en un mapeo creado con `mmap`, se debe realizar la traducción a dirección física de la siguiente manera:

- Verificamos si una dirección se encuentra dentro del rango de un *mapping*.
- Si no está dentro del rango, procedemos a traducir la memoria de manera normal. Sin embargo, si está dentro de un *mapping*, debemos:
 - Hallar el *cache frame* al que se encuentra mapeada la *page*.
 - Sumar el offset de la dirección base del *cache frame*.

Importante

Se les entregará una versión funcional de fork-wait. Si un proceso realiza `fork` después de haber creado un mapping, el proceso hijo debe heredar los mappings del proceso padre. Las páginas físicas del *page cache* no deben duplicarse. Tanto el padre como el hijo deben mapear sus páginas virtuales a los mismos *cache frames*.

5 Entregables

1. Carpeta .zip con:
 - Archivo `selfie.c`
 - Archivos de test
2. Tests que validen la correctitud de su implementación:
 - Al menos un test en el cuál se cree un *mapping* de un archivo y se lea su contenido directamente de memoria.
 - Al menos un test en el cuál se cree un *mapping* de un archivo y se escriba sobre este, mostrando posteriormente el cambio en memoria secundaria.
 - Al menos un test en el cuál dos procesos crean un *mapping del mismo archivo y lo modifican, siendo capaces de observar estos cambios en memoria*.
3. Presentación de 12 minutos donde presentan brevemente sus ideas, metodología y muestran una demo de sus tests. Las diapositivas también serán calificadas.

Importante

Se revisarán periódicamente sus avances durante sesiones de clase. Estas revisiones tomarán parte de la nota.